



UNIVERSITÀ  
DEGLI STUDI  
FIRENZE

Università degli Studi di Firenze

Scuola di Ingegneria  
Corso di Laurea in Ingegneria Informatica

## Laboratorio di Algoritmi

Confronto di differenti strategie per la gestione delle  
chiavi duplicate negli Alberi Binari di Ricerca

**Studente**

Etiona Cepele

**Matricola**

7169982

**Anno Accademico**

2025–2026

**Corso**

Algoritmi e strutture dati

**Professore**

Simone Marinai

# Indice

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                                      | <b>2</b>  |
| <b>2</b> | <b>Metodologia sperimentale</b>                          | <b>2</b>  |
| 2.1      | Piattaforma di test . . . . .                            | 2         |
| <b>3</b> | <b>Richiami teorici</b>                                  | <b>3</b>  |
| 3.1      | Alberi Binari di Ricerca . . . . .                       | 3         |
| 3.2      | Operazioni fondamentali e complessità . . . . .          | 4         |
| 3.3      | Problema della gestione delle chiavi duplicate . . . . . | 4         |
| <b>4</b> | <b>Descrizione delle implementazioni</b>                 | <b>5</b>  |
| 4.1      | Strategia StandardBST . . . . .                          | 5         |
| 4.2      | Strategia FlagBST . . . . .                              | 6         |
| 4.3      | Strategia ListBST . . . . .                              | 7         |
| <b>5</b> | <b>Dataset e procedura di test</b>                       | <b>8</b>  |
| 5.1      | Dataset utilizzati . . . . .                             | 8         |
| 5.2      | Procedura di benchmark . . . . .                         | 9         |
| <b>6</b> | <b>Risultati sperimentali</b>                            | <b>10</b> |
| 6.1      | Dataset casuale . . . . .                                | 10        |
| 6.2      | Dataset con duplicati moderati . . . . .                 | 12        |
| 6.3      | Dataset con molti duplicati . . . . .                    | 14        |
| <b>7</b> | <b>Discussione dei risultati</b>                         | <b>16</b> |
| <b>8</b> | <b>Conclusioni</b>                                       | <b>17</b> |
| <b>9</b> | <b>Bibliografia</b>                                      | <b>20</b> |

# 1 Introduzione

Gli Alberi Binari di Ricerca (Binary Search Tree, BST) rappresentano una delle strutture dati fondamentali per la gestione efficiente di insiemi dinamici di elementi ordinati.

La loro principale caratteristica consiste nella possibilità di effettuare operazioni come ricerca, inserimento ed eliminazione sfruttando la proprietà di ordinamento dei nodi: per ogni nodo, gli elementi presenti nel sottoalbero sinistro hanno chiave minore, mentre quelli presenti nel sottoalbero destro hanno chiave maggiore.

Nel caso in cui vengano inseriti elementi con chiavi duplicate, tuttavia, la definizione classica di BST non specifica un comportamento univoco. È quindi necessario adottare una strategia per rappresentare e gestire correttamente più occorrenze della stessa chiave, evitando, per quanto possibile, di compromettere le prestazioni della struttura.

Lo scopo di questo progetto è analizzare e confrontare tre differenti approcci per la gestione delle chiavi duplicate all'interno di un Albero Binario di Ricerca:

- **StandardBST**: inserimento delle chiavi duplicate seguendo una regola fissa, in questo caso posizionandole nel sottoalbero destro;
- **FlagBST**: inserimento delle chiavi duplicate come nuovi nodi, scegliendo alternativamente il sottoalbero sinistro o destro sulla base di un flag booleano associato al nodo;
- **ListBST**: gestione delle chiavi duplicate mediante una lista collegata associata al nodo corrispondente alla chiave.

Le tre implementazioni vengono valutate sperimentalmente attraverso diversi dataset, analizzando i tempi di esecuzione dell'operazione di inserimento, l'altezza degli alberi e il numero di nodi generati al variare della dimensione dell'input.

L'obiettivo finale è determinare quale strategia risulti più efficiente nei diversi scenari considerati e comprendere come la gestione delle duplicazioni influenzi il comportamento complessivo dell'albero.

## 2 Metodologia sperimentale

### 2.1 Piattaforma di test

Tutti gli esperimenti descritti nella presente relazione sono stati eseguiti utilizzando la medesima piattaforma hardware e software, in modo da garantire un confronto coerente tra le diverse implementazioni analizzate.

Le caratteristiche hardware principali del computer utilizzato sono riportate di seguito:

- **Processore**: AMD Ryzen 5 7520U with Radeon Graphics (2.80 GHz);

- **Memoria RAM:** 16,0 GB (15,3 GB utilizzabile);
- **Scheda grafica:** AMD Radeon(TM) Graphics (486 MB).

Il linguaggio di programmazione utilizzato per la realizzazione delle implementazioni e per l'esecuzione dei test è stato **Python**.

Il codice è stato sviluppato ed eseguito mediante l'ambiente di sviluppo **Visual Studio Code**.

La stesura della presente relazione è stata effettuata utilizzando l'editor **TeXworks** con il linguaggio di composizione tipografica  $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ .

## 3 Richiami teorici

### 3.1 Alberi Binari di Ricerca

Un Albero Binario di Ricerca (Binary Search Tree, BST) è una struttura dati gerarchica costituita da nodi organizzati secondo una specifica proprietà di ordinamento.

Ogni nodo dell'albero contiene una chiave e due riferimenti, rispettivamente al figlio sinistro e al figlio destro.

La proprietà fondamentale di un BST stabilisce che, per ogni nodo  $v$ :

- tutte le chiavi presenti nel sottoalbero sinistro di  $v$  sono minori della chiave contenuta nel nodo  $v$ ;
- tutte le chiavi presenti nel sottoalbero destro di  $v$  sono maggiori della chiave contenuta nel nodo  $v$ .

Formalmente, indicando con  $k(v)$  la chiave associata al nodo  $v$ , si ha:

$$\forall x \in \text{left}(v), \quad k(x) < k(v)$$

e

$$\forall y \in \text{right}(v), \quad k(y) > k(v).$$

Questa proprietà permette di effettuare la ricerca di un elemento senza dover analizzare tutti i nodi dell'albero.

A ogni confronto, infatti, è possibile stabilire se proseguire la ricerca nel sottoalbero sinistro oppure in quello destro, riducendo progressivamente lo spazio di ricerca.

## 3.2 Operazioni fondamentali e complessità

Le principali operazioni definite su un Albero Binario di Ricerca sono:

- **Ricerca:** permette di verificare se una determinata chiave è presente nell'albero;
- **Inserimento:** consente di aggiungere un nuovo elemento mantenendo invariata la proprietà di ordinamento;
- **Cancellazione:** permette di rimuovere un nodo dall'albero mantenendo la struttura valida.

La complessità computazionale di tali operazioni dipende dall'altezza  $h$  dell'albero. Nel caso generale, ricerca, inserimento ed eliminazione hanno complessità:

$$O(h)$$

dove  $h$  rappresenta il numero massimo di archi presenti lungo un cammino dalla radice alla foglia più distante.

Nel caso ideale, quando l'albero è bilanciato, l'altezza assume valore:

$$h = O(\log n)$$

dove  $n$  rappresenta il numero di nodi presenti nell'albero.

Di conseguenza, le operazioni principali possono essere eseguite in tempo logaritmico.

Tuttavia, nel caso peggiore, un BST può assumere una forma completamente sbilanciata, simile a una lista concatenata.

In questa situazione l'altezza diventa:

$$h = O(n)$$

e le operazioni degradano a una complessità lineare.

## 3.3 Problema della gestione delle chiavi duplicate

La definizione classica di Albero Binario di Ricerca assume generalmente che tutte le chiavi siano distinte.

Quando vengono inseriti elementi duplicati, però, la proprietà di ordinamento non specifica dove debbano essere posizionati gli elementi aventi la stessa chiave.

È quindi necessario introdurre una strategia aggiuntiva per gestire correttamente le duplicazioni.

Le principali possibilità sono:

1. inserire le chiavi duplicate come nuovi nodi, adottando una regola fissa, ad esempio sempre nel sottoalbero destro;
2. inserire le chiavi duplicate come nuovi nodi, scegliendo alternativamente il sottoalbero sinistro o destro mediante un'informazione aggiuntiva associata al nodo;
3. associare a ogni nodo una struttura dati ausiliaria contenente le occorrenze duplicate della stessa chiave.

Nel presente progetto vengono analizzate proprio queste tre strategie attraverso le implementazioni **StandardBST**, **FlagBST** e **ListBST**, con l'obiettivo di confrontarne vantaggi e svantaggi in termini di efficienza e comportamento strutturale.

Una differenza fondamentale tra le tre strategie riguarda il modo in cui vengono rappresentate le occorrenze duplicate. **StandardBST** e **FlagBST** creano un nuovo nodo per ogni elemento inserito, mentre **ListBST** raggruppa le occorrenze della stessa chiave all'interno di una struttura ausiliaria.

Per semplicità, nell'ambito di questo progetto non viene analizzata l'operazione di cancellazione dei nodi.

Lo studio si concentra principalmente sull'operazione di inserimento, poiché è quella maggiormente influenzata dalla scelta della strategia adottata per la gestione delle chiavi duplicate.

## 4 Descrizione delle implementazioni

Le tre strategie permettono di confrontare approcci differenti alla gestione delle chiavi duplicate.

La **StandardBST** mantiene la struttura tradizionale dell'ABR e utilizza una regola fissa per i duplicati. La **FlagBST** mantiene anch'essa ogni occorrenza come nodo separato, ma introduce un meccanismo per distribuire alternativamente i duplicati nei due sottoalberi. La **ListBST**, invece, raggruppa le occorrenze duplicate in una struttura ausiliaria associata al nodo.

### 4.1 Strategia StandardBST

La prima implementazione considera ogni occorrenza di una chiave duplicata come un nuovo nodo indipendente dell'albero.

Nel caso in cui durante l'inserimento venga incontrato un nodo avente la stessa chiave dell'elemento da inserire, il nuovo nodo viene posizionato nel sottoalbero destro. Questa scelta permette di definire una regola deterministica per la gestione dei duplicati mantenendo invariata la struttura classica di un ABR.

Ogni nodo contiene solamente:

- la chiave memorizzata;
- il riferimento al figlio sinistro;
- il riferimento al figlio destro.

Il vantaggio principale di questa soluzione è rappresentato dalla semplicità implementativa, poiché non richiede modifiche sostanziali alla struttura tradizionale del BST.

Tuttavia, quando il dataset contiene molte chiavi duplicate, l'inserimento ripetuto degli stessi valori può provocare un aumento eccessivo dell'altezza dell'albero.

In particolare, la presenza di numerose occorrenze dello stesso valore può generare una struttura fortemente sbilanciata, con un conseguente peggioramento delle prestazioni delle operazioni di ricerca e inserimento.

Nel caso di numerosi duplicati, infatti, ogni nuova occorrenza viene aggiunta lungo una catena di nodi appartenenti al sottoalbero destro, determinando una crescita quasi lineare dell'altezza.

## 4.2 Strategia FlagBST

La seconda strategia analizzata mantiene ogni occorrenza come un nodo distinto, ma introduce all'interno del nodo un flag booleano utilizzato per stabilire la direzione da seguire quando viene incontrata una chiave duplicata.

Ogni nodo della **FlagBST** contiene quindi, oltre alla chiave e ai riferimenti ai figli, un campo booleano denominato **flag**.

Il flag viene utilizzato secondo la seguente regola:

- se il flag è **False**, il successivo duplicato incontrato viene indirizzato verso il sottoalbero sinistro;
- se il flag è **True**, il successivo duplicato viene indirizzato verso il sottoalbero destro;
- dopo ogni visita del nodo con una chiave uguale, il valore del flag viene invertito.

In questo modo, a differenza della StandardBST, le chiavi duplicate non vengono indirizzate sempre nella stessa direzione.

È importante sottolineare che il flag non rappresenta il numero di occorrenze della chiave e non permette di memorizzare più duplicati all'interno dello stesso nodo. Ogni elemento inserito continua infatti a corrispondere a un nuovo nodo dell'albero.

Durante l'operazione di inserimento, l'algoritmo procede inizialmente come un normale BST:

- se la chiave da inserire è minore della chiave del nodo corrente, la ricerca prosegue nel sottoalbero sinistro;

- se la chiave da inserire è maggiore, la ricerca prosegue nel sottoalbero destro;
- se la chiave coincide con quella del nodo corrente, viene consultato il flag per determinare la direzione da seguire.

Quando la direzione scelta dal flag conduce a un figlio già esistente, l'inserimento continua a partire da quel figlio, applicando nuovamente le normali regole dell'ABR.

Il meccanismo ha quindi lo scopo di distribuire le occorrenze duplicate tra i due sottoalberi, evitando la formazione della lunga catena unidirezionale tipica della StandardBST quando lo stesso valore viene inserito ripetutamente.

Il vantaggio principale della FlagBST è quindi di natura strutturale: pur mantenendo un nodo separato per ogni occorrenza, permette di limitare fortemente la crescita dell'altezza dell'albero nei dataset caratterizzati da numerosi duplicati.

Questo comportamento, come mostrato dai risultati sperimentali, può produrre un miglioramento significativo dei tempi di inserimento rispetto alla StandardBST.

Tuttavia, la strategia non riduce il numero di nodi e non costituisce una garanzia formale di bilanciamento logaritmico per qualunque sequenza di inserimento. Il miglioramento deve quindi essere interpretato come un effetto della distribuzione alternata dei duplicati osservato sperimentalmente.

### 4.3 Strategia ListBST

L'ultima strategia considerata consiste nell'associare a ogni nodo dell'albero una struttura ausiliaria contenente le occorrenze duplicate della stessa chiave.

In questa implementazione, ogni chiave distinta viene rappresentata da un singolo nodo dell'albero. Le ulteriori occorrenze della stessa chiave vengono invece memorizzate all'interno di una lista collegata associata al nodo.

Quando viene inserita una nuova chiave:

- se il valore non è presente nell'albero, viene creato un nuovo nodo secondo le regole classiche di un BST;
- se il valore coincide con una chiave già presente, la nuova occorrenza viene aggiunta alla lista associata al nodo esistente.

Rispetto alla FlagBST, questa soluzione non crea nuovi nodi dell'albero per le occorrenze duplicate.

La struttura del nodo risulta quindi composta da:

- la chiave del nodo;
- il riferimento al figlio sinistro;

- il riferimento al figlio destro;
- una lista contenente le occorrenze duplicate della chiave.

Il principale vantaggio della ListBST è la riduzione del numero di nodi dell'albero quando sono presenti molte duplicazioni.

Di conseguenza, anche l'altezza può risultare notevolmente inferiore rispetto alle strategie che creano un nuovo nodo per ogni occorrenza.

Lo svantaggio consiste nell'utilizzo di memoria aggiuntiva dovuto alla gestione delle liste associate ai nodi. Inoltre, se fosse necessario accedere alle singole occorrenze duplicate, il costo dell'operazione dipenderebbe dalla struttura utilizzata per rappresentarle.

## 5 Dataset e procedura di test

### 5.1 Dataset utilizzati

Per valutare il comportamento delle tre implementazioni sono stati utilizzati tre differenti dataset, generati tramite funzioni Python appositamente definite.

La scelta dei dataset è stata effettuata con l'obiettivo di analizzare il comportamento delle strutture al variare della frequenza con cui compaiono chiavi duplicate.

In particolare, sono stati considerati i seguenti casi:

- **Dataset casuale** (`random_dataset`):

contiene  $n$  valori generati casualmente nell'intervallo  $[0, 10n)$ .

La generazione viene effettuata tramite campionamento senza ripetizione, garantendo quindi l'assenza di duplicati all'interno del dataset.

Questo caso rappresenta lo scenario in cui l'albero viene costruito con chiavi distinte.

- **Dataset con duplicati moderati** (`duplicated_dataset`):

contiene  $n$  valori generati casualmente nell'intervallo intero

$$0, \dots, \left\lfloor \frac{n}{2} \right\rfloor.$$

Poiché lo spazio dei possibili valori è inferiore rispetto al numero di elementi da inserire, possono verificarsi diverse occorrenze della stessa chiave.

Questo dataset permette di osservare il comportamento delle strategie in presenza di una quantità moderata di duplicazioni.

- **Dataset con molti duplicati** (`many_duplicates_dataset`):

contiene  $n$  valori generati casualmente nell'intervallo

$0, \dots, 10$ .

In questo caso sono presenti solamente 11 possibili chiavi, producendo un'elevata concentrazione di duplicati all'aumentare di  $n$ .

Questo scenario permette di evidenziare maggiormente le differenze tra le strategie adottate per la gestione delle chiavi ripetute.

Per ciascun dataset sono state considerate diverse dimensioni dell'input, in modo da analizzare l'evoluzione dei tempi di esecuzione all'aumentare del numero di elementi inseriti.

Le dimensioni utilizzate sono:

$$n \in \{1000, 5000, 10000, 50000\}.$$

## 5.2 Procedura di benchmark

Il confronto sperimentale tra le tre implementazioni è stato effettuato mediante un programma di benchmark sviluppato in Python.

Per ogni dataset generato e per ogni dimensione dell'input considerata, viene creato un nuovo albero inizialmente vuoto.

Successivamente tutti gli elementi appartenenti al dataset vengono inseriti nella struttura attraverso il metodo `insert()`.

Il tempo necessario alla costruzione dell'albero viene misurato utilizzando la funzione `time.perf_counter()`, che permette di ottenere una misura ad alta precisione degli intervalli temporali.

La procedura di test è stata applicata alle tre implementazioni considerate:

- **StandardBST**;
- **FlagBST**;
- **ListBST**.

Per ogni combinazione tra dataset, dimensione dell'input e implementazione sono state raccolte diverse metriche:

- **Tempo di inserimento**: rappresenta il tempo totale necessario per inserire tutti gli elementi del dataset nell'albero.
- **Altezza dell'albero**: indica la lunghezza del cammino massimo dalla radice alla foglia più distante e permette di valutare il livello di bilanciamento raggiunto dalla struttura.

- **Numero di nodi:** rappresenta il numero effettivo di nodi creati all'interno dell'albero.
- **Numero totale di elementi:** indica il numero complessivo di valori memorizzati nella struttura, includendo le occorrenze duplicate.

Per StandardBST e FlagBST, ogni occorrenza viene rappresentata da un nodo distinto e quindi, al termine dell'inserimento, il numero di nodi coincide con il numero totale di elementi.

Per ListBST, invece, le occorrenze duplicate vengono memorizzate nella lista associata al nodo e quindi il numero di nodi coincide con il numero di chiavi distinte, mentre il numero totale di elementi rimane pari alla dimensione del dataset.

I risultati ottenuti sono stati salvati in un file CSV, successivamente utilizzato per l'analisi e la generazione dei grafici comparativi.

## 6 Risultati sperimentali

In questa sezione vengono presentati e analizzati i risultati ottenuti durante la fase di benchmark.

Il confronto viene effettuato considerando le tre implementazioni sviluppate:

- StandardBST;
- FlagBST;
- ListBST.

Per ogni dataset vengono analizzati il tempo di inserimento, l'altezza dell'albero e il numero di nodi generati, al fine di valutare sia le prestazioni computazionali sia l'effetto della gestione delle chiavi duplicate sulla struttura dell'albero.

### 6.1 Dataset casuale

Il primo esperimento considera il dataset **random**, caratterizzato dalla generazione di valori casuali senza ripetizione.

In questo scenario non sono presenti chiavi duplicate e quindi le tre implementazioni si trovano nella condizione in cui la gestione delle duplicazioni non influisce sulla costruzione dell'albero.

I risultati mostrano infatti che tutte le strategie producono alberi aventi la stessa struttura.

Per ogni dimensione del dataset, il numero di nodi generati coincide con il numero totale di elementi inseriti e l'altezza dell'albero risulta identica per le tre implementazioni.

Nel caso con  $n = 50000$  elementi, tutte e tre le implementazioni producono:

- un albero di altezza pari a 37;
- 50000 nodi;
- 50000 elementi complessivi.

La differenza principale riguarda il tempo di inserimento.

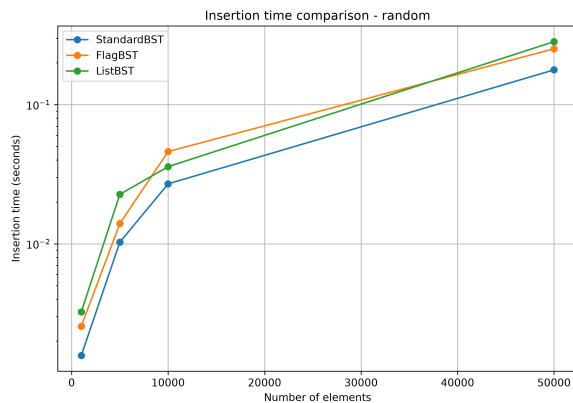
Per  $n = 50000$  si ottengono i seguenti valori:

- StandardBST: circa 0.149 secondi;
- FlagBST: circa 0.210 secondi;
- ListBST: circa 0.258 secondi.

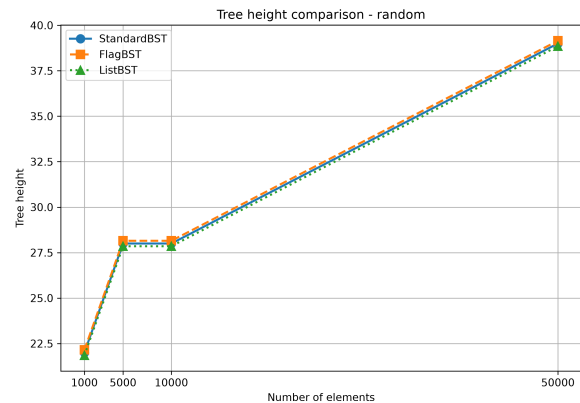
La strategia **StandardBST** risulta quindi la più veloce nel caso di dati privi di duplicati.

Questo risultato è coerente con le caratteristiche delle implementazioni: quando non vengono mai incontrate chiavi uguali, il meccanismo di gestione dei duplicati della FlagBST non viene sfruttato e la gestione della lista della ListBST non porta alcun vantaggio strutturale.

Le strutture aggiuntive presenti nelle due strategie introducono invece un piccolo overhead durante l'esecuzione.



(a) Tempo di inserimento



(b) Altezza degli alberi

Figura 1: Confronto del tempo di inserimento e dell'altezza degli alberi per il dataset casuale.

## 6.2 Dataset con duplicati moderati

Il secondo esperimento è stato condotto utilizzando il dataset `medium_duplicates`, nel quale i valori vengono generati casualmente nell'intervallo

$$0, \dots, \left\lfloor \frac{n}{2} \right\rfloor.$$

Tale scelta comporta la presenza di un numero moderato di chiavi duplicate, permettendo di valutare l'effetto delle diverse strategie sulla struttura dell'albero.

A differenza del dataset casuale, in questo caso le tre implementazioni non producono più alberi equivalenti.

Sia **StandardBST** sia **FlagBST** creano infatti un nuovo nodo per ogni elemento inserito. La differenza consiste nella direzione scelta quando viene incontrata una chiave duplicata: StandardBST procede sempre verso destra, mentre FlagBST alterna la direzione sulla base del flag booleano.

I risultati relativi a  $n = 50000$  mostrano che:

- StandardBST crea 50000 nodi e raggiunge un'altezza pari a 41;
- FlagBST crea 50000 nodi e raggiunge anch'essa un'altezza pari a 41;
- ListBST crea 21666 nodi e raggiunge un'altezza pari a 38.

Il numero di nodi della ListBST è quindi sensibilmente inferiore, poiché le occorrenze duplicate vengono raggruppate all'interno delle liste associate ai nodi.

Per StandardBST e FlagBST, invece, il numero di nodi coincide con il numero totale di elementi inseriti.

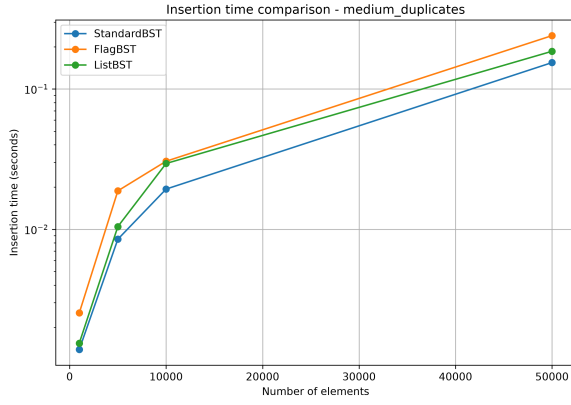
Per quanto riguarda l'altezza, il vantaggio della FlagBST rispetto alla StandardBST in questo dataset non risulta sistematico.

Considerando tutte le dimensioni testate, le altezze ottenute sono:

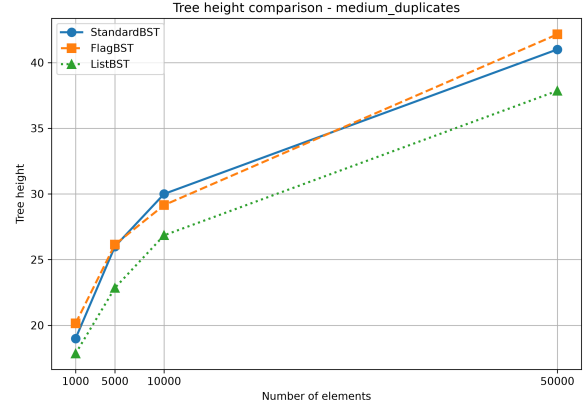
| $n$   | StandardBST | FlagBST | ListBST |
|-------|-------------|---------|---------|
| 1000  | 22          | 21      | 17      |
| 5000  | 27          | 29      | 25      |
| 10000 | 27          | 26      | 24      |
| 50000 | 41          | 41      | 38      |

Si osserva quindi che FlagBST può risultare leggermente migliore o peggiore della StandardBST a seconda della distribuzione effettiva dei valori nel dataset.

Questo comportamento è coerente con la natura della strategia: il flag cerca di distribuire i duplicati tra i due lati del nodo, ma non costituisce un algoritmo di bilanciamento generale dell'albero.



(a) Tempo di inserimento



(b) Altezza degli alberi

Figura 2: Confronto del tempo di inserimento e dell'altezza degli alberi per il dataset con duplicati moderati.

Dal punto di vista delle prestazioni temporali, in questo scenario la **StandardBST** risulta la più veloce tra le tre strategie considerate.

Nel caso con  $n = 50000$  elementi si ottengono:

- StandardBST: circa 0.147 secondi;
- ListBST: circa 0.159 secondi;
- FlagBST: circa 0.192 secondi.

La FlagBST presenta quindi un overhead maggiore rispetto alla StandardBST, dovuto ai controlli e all'aggiornamento del flag necessari ogni volta che viene incontrata una chiave duplicata.

La ListBST presenta invece un tempo intermedio: la riduzione del numero di nodi dell'albero compensa in parte il costo della gestione delle liste.

Nel complesso, il dataset con duplicati moderati non evidenzia un vantaggio netto della FlagBST rispetto alla StandardBST.

Il principale effetto positivo della strategia a flag diventa invece evidente nel caso caratterizzato da una concentrazione molto elevata di duplicati, analizzato nella sezione successiva.

## 6.3 Dataset con molti duplicati

L'ultimo esperimento è stato eseguito utilizzando il dataset `many_duplicates`, nel quale tutti i valori vengono generati casualmente nell'intervallo

$$0, \dots, 10.$$

Poiché il numero di possibili chiavi è estremamente ridotto rispetto alla dimensione del dataset, il numero di occorrenze duplicate risulta molto elevato.

Questo scenario rappresenta il caso in cui la scelta della strategia di gestione delle chiavi duplicate assume la maggiore importanza.

La differenza fondamentale tra le tre implementazioni è evidente nella struttura risultante.

La **StandardBST** crea un nuovo nodo per ogni elemento e inserisce sempre i duplicati verso destra.

La **FlagBST** crea anch'essa un nuovo nodo per ogni elemento, ma alterna la direzione dei duplicati mediante il flag booleano.

La **ListBST**, invece, mantiene un solo nodo per ogni chiave distinta e memorizza le occorrenze successive nella lista associata.

Nel caso con  $n = 50000$  elementi, i risultati sono:

- StandardBST: 50000 nodi, altezza 4641;
- FlagBST: 50000 nodi, altezza 28;
- ListBST: 11 nodi, altezza 4.

Il risultato più significativo riguarda quindi la differenza tra StandardBST e FlagBST.

Pur utilizzando lo stesso numero di nodi, la FlagBST riesce a evitare la crescita estremamente sbilanciata della StandardBST.

L'altezza della StandardBST passa infatti da 120 per  $n = 1000$  a 4641 per  $n = 50000$ , mentre quella della FlagBST passa solamente da 18 a 28.

I valori misurati sono riportati nella seguente tabella:

| $n$   | StandardBST | FlagBST | ListBST |
|-------|-------------|---------|---------|
| 1000  | 120         | 18      | 6       |
| 5000  | 474         | 22      | 4       |
| 10000 | 962         | 26      | 7       |
| 50000 | 4641        | 28      | 4       |

La FlagBST mostra quindi sperimentalmente un comportamento molto più equilibrato rispetto alla StandardBST in presenza di molti duplicati.

È importante sottolineare che questo risultato è di natura sperimentale: la strategia a flag non fornisce una garanzia teorica di altezza  $O(\log n)$  per ogni possibile sequenza di inserimento.

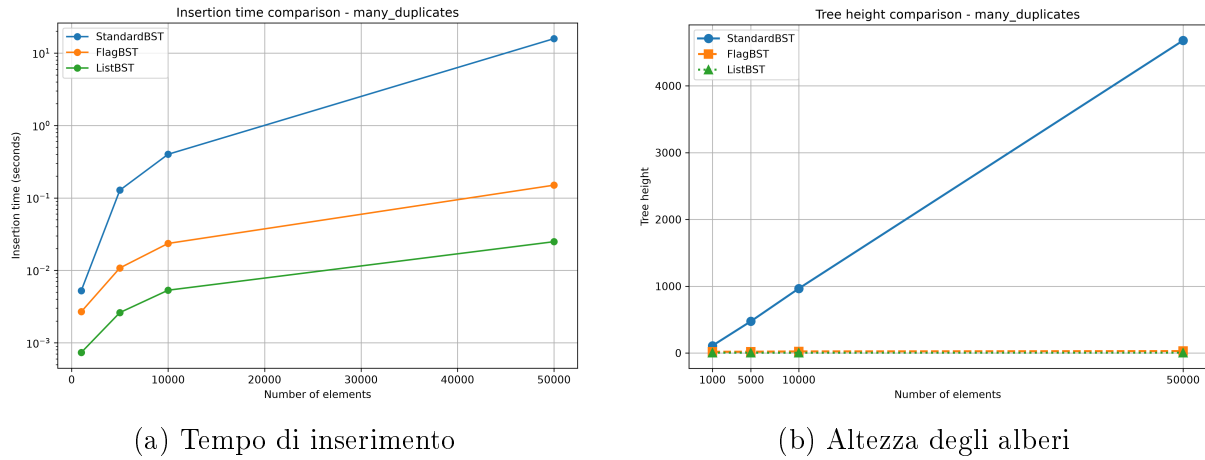


Figura 3: Confronto del tempo di inserimento e dell'altezza degli alberi per il dataset con molti duplicati.

Il grafico dei tempi di inserimento mostra chiaramente il vantaggio della FlagBST rispetto alla StandardBST.

I tempi misurati sono:

| $n$   | StandardBST | FlagBST  | ListBST  |
|-------|-------------|----------|----------|
| 1000  | 0.004293    | 0.001930 | 0.000905 |
| 5000  | 0.116021    | 0.010618 | 0.004166 |
| 10000 | 0.354764    | 0.024716 | 0.005632 |
| 50000 | 12.178867   | 0.144767 | 0.022933 |

Nel caso con  $n = 50000$ , la StandardBST richiede circa 12.18 secondi, mentre la FlagBST richiede circa 0.145 secondi.

La differenza è quindi molto significativa: la FlagBST risulta circa 84 volte più veloce della StandardBST in questo esperimento.

La ListBST ottiene il risultato migliore, con un tempo di circa 0.023 secondi.

Il suo vantaggio deriva dal fatto che l'albero contiene solamente 11 nodi, uno per ciascuna delle possibili chiavi del dataset.

## 7 Discussione dei risultati

L'analisi sperimentale ha evidenziato come l'efficacia delle tre strategie dipenda fortemente dalla distribuzione delle chiavi presenti nel dataset.

Nel caso del dataset casuale, caratterizzato dall'assenza di duplicati, tutte le implementazioni producono alberi strutturalmente equivalenti. L'altezza dell'albero e il numero di nodi coincidono in tutti gli esperimenti.

Le differenze nei tempi di esecuzione sono dovute principalmente all'overhead introdotto dalle rispettive implementazioni. La StandardBST risulta infatti la più veloce, mentre FlagBST e ListBST presentano tempi leggermente superiori.

In questo scenario, quindi, non vi è alcun vantaggio nell'utilizzare strategie specifiche per la gestione dei duplicati.

La situazione diventa più interessante nel dataset con duplicati moderati.

In questo caso StandardBST e FlagBST continuano a creare un nuovo nodo per ogni elemento, mentre ListBST raggruppa le occorrenze duplicate.

La FlagBST non presenta però un miglioramento sistematico rispetto alla StandardBST: in alcuni casi l'altezza risulta leggermente inferiore, mentre in altri risulta maggiore o uguale.

Anche dal punto di vista temporale, nel dataset analizzato la StandardBST risulta più veloce della FlagBST.

Questo risultato mostra che il semplice utilizzo di un flag non è sufficiente, in presenza di una quantità moderata di duplicati, a garantire un vantaggio prestazionale.

Il comportamento cambia radicalmente nel dataset con molti duplicati.

In questo caso la StandardBST presenta una forte degenerazione strutturale. L'inserimento ripetuto delle stesse chiavi sempre verso destra determina una crescita molto rapida dell'altezza.

Per  $n = 50000$ , l'altezza raggiunge infatti 4641 e il tempo di inserimento supera i 12 secondi.

La FlagBST mostra invece un comportamento molto diverso. Pur mantenendo 50000 nodi, riesce a distribuire le occorrenze duplicate tra i due sottoalberi grazie all'alternanza determinata dal flag.

L'altezza rimane quindi contenuta, raggiungendo un valore di 28 per  $n = 50000$ .

Questa differenza strutturale si riflette direttamente sul tempo di inserimento: la FlagBST completa l'inserimento degli stessi 50000 elementi in circa 0.145 secondi, risultando molto più veloce della StandardBST.

La ListBST rappresenta invece la soluzione più compatta nei dataset con molti duplicati.

Nel caso con  $n = 50000$ , sono sufficienti 11 nodi, corrispondenti alle 11 possibili chiavi, mentre tutte le occorrenze vengono memorizzate nelle liste associate.

La riduzione del numero di nodi permette di ottenere anche l'altezza più bassa e il tempo di inserimento migliore tra le tre strategie.

È quindi possibile individuare una differenza fondamentale tra FlagBST e ListBST.

La FlagBST cerca di migliorare la distribuzione strutturale dei duplicati mantenendo ogni occorrenza come nodo separato.

La ListBST, invece, affronta il problema alla radice riducendo il numero di nodi dell'albero e spostando la gestione delle occorrenze duplicate in una struttura ausiliaria.

Dal punto di vista della memoria, pertanto, le due strategie presentano caratteristiche differenti. FlagBST utilizza un nodo per ogni elemento, aggiungendo solamente un'informazione booleana, mentre ListBST riduce il numero di nodi dell'albero ma necessita di memoria aggiuntiva per le liste.

Nel complesso, i risultati mostrano che non esiste una strategia universalmente migliore.

La StandardBST è particolarmente adatta quando i duplicati sono assenti o poco rilevanti e si desidera mantenere la struttura più semplice possibile.

La FlagBST rappresenta invece un compromesso interessante quando si desidera mantenere ogni occorrenza come nodo separato ma si vuole evitare la forte degenerazione provocata dall'inserimento unidirezionale dei duplicati.

La ListBST risulta infine particolarmente efficace quando il numero di duplicati è elevato, poiché permette di mantenere un albero molto più compatto.

## 8 Conclusioni

L'obiettivo di questo progetto era confrontare differenti strategie per la gestione delle chiavi duplicate negli Alberi Binari di Ricerca, analizzandone il comportamento sia dal punto di vista strutturale sia dal punto di vista delle prestazioni.

Le implementazioni sviluppate hanno mostrato come la scelta della strategia adottata influenzi significativamente l'efficienza dell'albero in funzione della distribuzione delle chiavi presenti nei dataset.

Nel caso di dati privi di duplicati, la strategia StandardBST rappresenta la soluzione più semplice ed efficiente.

Le tre implementazioni producono infatti alberi con la stessa altezza e lo stesso numero di nodi, mentre StandardBST ottiene i tempi di inserimento migliori grazie alla maggiore semplicità della struttura.

Quando compaiono duplicati, le differenze tra le strategie diventano invece più significative.

La StandardBST continua a creare un nuovo nodo per ogni occorrenza e inserisce sempre i duplicati nel sottoalbero destro. Questa scelta è semplice da implementare,

ma può provocare una forte degenerazione dell'albero quando una stessa chiave compare numerose volte.

La FlagBST adotta un approccio differente: ogni occorrenza continua a essere rappresentata da un nodo separato, ma la direzione scelta per un duplicato viene alternata mediante un flag booleano memorizzato nel nodo.

Il vantaggio principale di questa strategia non consiste quindi nella riduzione del numero di nodi, bensì nella distribuzione più equilibrata delle occorrenze duplicate.

I risultati ottenuti nel dataset con molti duplicati mostrano chiaramente l'efficacia di questo approccio. Con 50000 elementi, StandardBST raggiunge un'altezza di 4641 e richiede circa 12.18 secondi per completare gli inserimenti, mentre FlagBST raggiunge un'altezza di soli 28 e richiede circa 0.145 secondi.

La FlagBST risulta quindi particolarmente efficace nel prevenire la degenerazione dell'albero causata da una lunga sequenza di duplicati.

È tuttavia importante precisare che tale strategia non fornisce una garanzia teorica di bilanciamento logaritmico per ogni possibile sequenza di inserimento. I risultati osservati devono quindi essere interpretati come un'evidenza sperimentale dell'efficacia del meccanismo di alternanza.

La ListBST ottiene invece i risultati migliori nei dataset caratterizzati da un'elevata concentrazione di duplicati.

Nel caso con 50000 elementi e soli 11 valori possibili, l'albero contiene solamente 11 nodi e raggiunge un'altezza pari a 4, con un tempo di inserimento di circa 0.023 secondi.

Questo risultato è dovuto alla possibilità di rappresentare tutte le occorrenze della stessa chiave mediante una singola struttura ausiliaria, evitando di aumentare il numero di nodi dell'albero.

La ListBST richiede però una gestione aggiuntiva delle liste e quindi un'organizzazione della memoria più complessa rispetto alla StandardBST e alla FlagBST.

In conclusione, le tre strategie presentano vantaggi differenti.

La **StandardBST** privilegia la semplicità ed è adeguata quando i duplicati sono assenti.

La **FlagBST** rappresenta un compromesso tra semplicità e robustezza strutturale: mantiene ogni occorrenza come nodo separato, ma attraverso l'alternanza del flag evita, nei casi sperimentati, la forte degenerazione prodotta dalla gestione unidirezionale dei duplicati.

La **ListBST** risulta invece la soluzione più efficace quando la quantità di duplicati è elevata, poiché riduce drasticamente il numero di nodi e l'altezza dell'albero.

Il progetto ha quindi evidenziato come la gestione delle chiavi duplicate non sia solamente un dettaglio implementativo, ma possa influenzare in maniera sostanziale la struttura dell'albero e le prestazioni delle operazioni su di esso.

La scelta della strategia più adatta deve pertanto essere effettuata considerando le caratteristiche dei dati da memorizzare e, in particolare, la frequenza con cui sono presenti chiavi duplicate.

## 9 Bibliografia

### Riferimenti bibliografici

- [1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein,  
*Introduction to Algorithms*,  
MIT Press, Fourth Edition, 2022.
- [2] Python Software Foundation,  
*Python 3 Documentation*,  
<https://docs.python.org/3/>